

第3章 程序设计与算法思维

计算机语言与程序

邹 华

武汉大学计算机学院

第3章 程序设计与算法思维

2

一、计算机语言的发展

二、不同的计算机语言之异同点

三、程序构造：控制结构

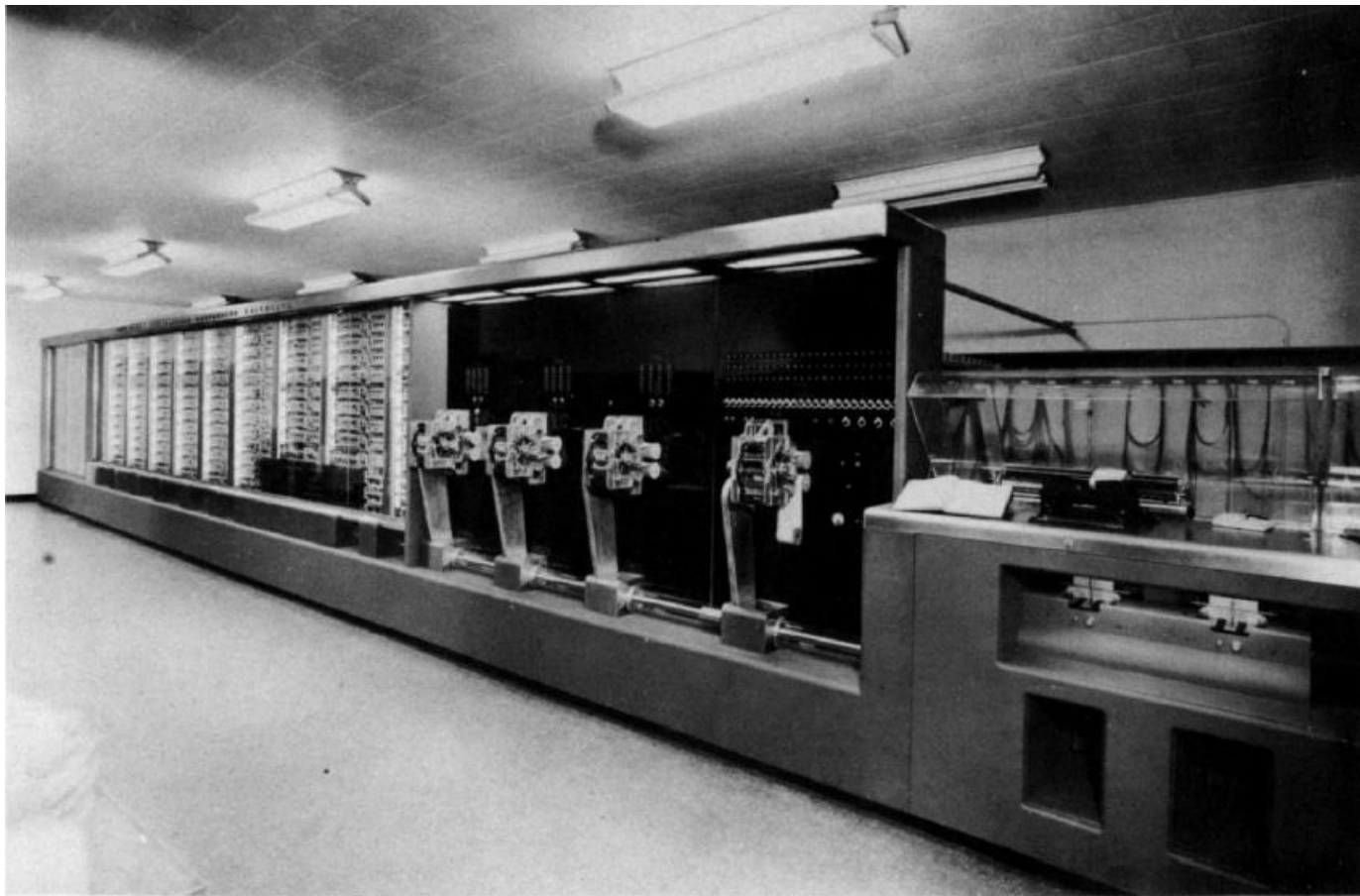
四、程序构造：函数、递归与迭代

五、算法表达与思维

计算机语言的发展

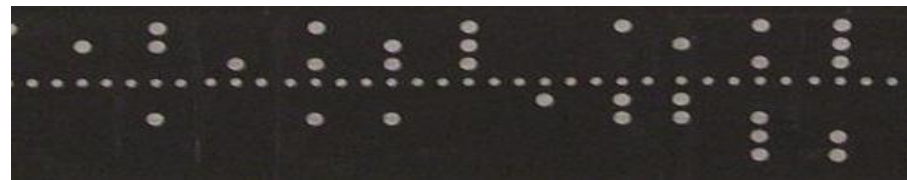
3

起源



1944年 Harvard Mark I

- ✓ Mark I是当年的大型计算机，由约765000个机电元件组成。长约15.5米，高约2.4米，重达5吨
- ✓ 机器左侧是置数旋钮阵列，可输入60个23位十进制数
- ✓ 中间部分是计算阵列，可存放72个23位十进制数
- ✓ 机器的右侧部分是若干台穿孔式输入输出装置



输入输出纸带

计算机语言的发展

起源

计数器编号

序号	编号	序号	编号	序号	编号	序号	编号	序号	编号	序号	编号
C1	1	C13	431	C25	541	C37	631	C49	651	C61	65431
C2	2	C14	432	C26	542	C38	632	C50	652	C62	65432
C3	21	C15	4321	C27	5421	C39	6321	C51	6521	C63	654321
C4	3	C16	5	C28	543	C40	64	C52	653	C64	7
C5	31	C17	51	C29	5431	C41	641	C53	6531	C65	71
C6	32	C18	52	C30	5432	C42	642	C54	6532	C66	72
C7	321	C19	521	C31	54321	C43	6421	C55	65321	C67	721
C8	4	C20	53	C32	6	C44	643	C56	654	C68	73
C9	41	C21	531	C33	61	C45	6431	C57	6541	C69	731
C10	42	C22	532	C34	62	C46	6432	C58	6542	C70	732
C11	421	C23	5321	C35	621	C47	64321	C59	65421	C71	7321
C12	43	C24	54	C36	63	C48	65	C60	6543	C72	74

置数旋钮组编号

序号	编号	序号	编号	序号	编号	序号	编号	序号	编号	序号	编号
S1	741	S11	7521	S21	75431	S31	76321	S41	7651	S51	765421
S2	742	S12	753	S22	75432	S32	764	S42	7652	S52	76543
S3	7421	S13	7531	S23	754321	S33	7641	S43	76521	S53	765431
S4	743	S14	7532	S24	76	S34	7642	S44	7653	S54	765432
S5	7431	S15	75321	S25	761	S35	76421	S45	76531	S55	7654321
S6	7432	S16	754	S26	762	S36	7643	S46	76532	S56	8
S7	74321	S17	7541	S27	7621	S37	76431	S47	765321	S57	81
S8	75	S18	7542	S28	763	S38	76432	S48	7654	S58	82
S9	751	S19	75421	S29	7631	S39	764321	S49	76541	S59	821
S10	752	S20	7543	S30	7632	S40	765	S50	76542	S60	83

操作	编程代码	穿孔
S10+C24 (结果存入C24)	752 54 7	○●○○●○○○●○○○●●○○○●○○○○○○○

$C24 = S10 + C24$

编号：752

编号：54

顺序号：82

顺序号：24

01010010

00011000

○●○○●○○○●○○

○○○●●○○○○

最左侧纸带

中间纸带

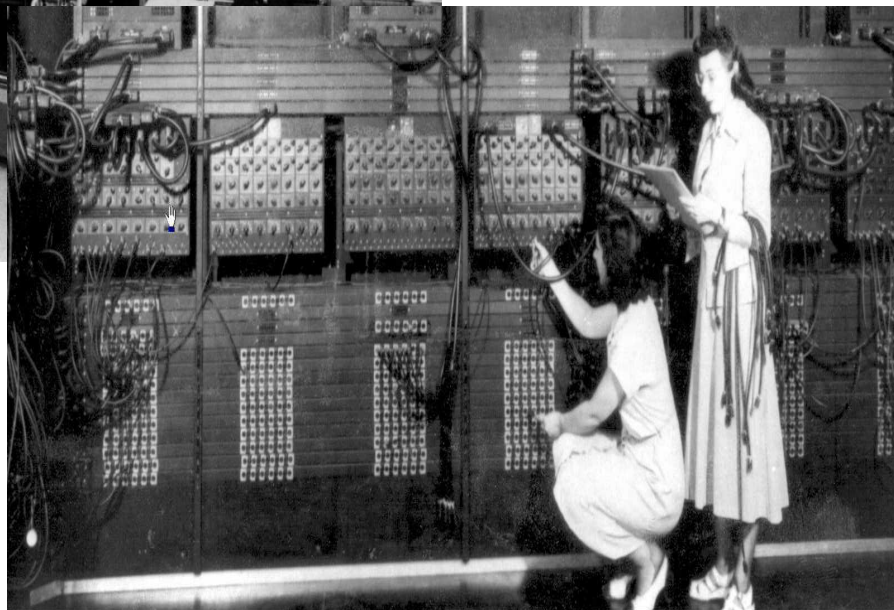
计算机语言的发展

5

起源



Harvard Mark I



1946年 ENIAC

- ✓ 计算每位同学的成绩绩点
 - ✓ 对全班同学的成绩进行排序
 - ✓ 按成绩排名比例划分奖学金等级
 - ✓
-
- ✓ 纸带不便于修改
 - ✓ 出错后需要拼接纸带
 - ✓ 不便于运输、保存
 - ✓ 纸带消耗巨大

计算机语言的发展

6

算法、计算机语言与计算机程序

步骤书写的规范、语法规则、标准的集合
是人和计算机都能理解的语言

高级语言
语法要素

程序设计
技巧

程序设计
环境

读程序
写程序
练

算法

计算机语言

程序

解决问题
的步骤

计算机能够理解与
执行的解决问题的步骤

算法设计
策略

数据结构
设计

算法分析
方法

算法表达
方法

读算法
写算法
练

计算机语言的发展

7

机器语言与机器程序

指令
系统

CPU用二进制和编码提供的可以解释并执行的命令的集合

操作码

地址码

100001	10 00000111
100010	11 00001010

机器
语言

用二进制和编码方式提供的指令系统所编写程序的语言

计算7+10并存储的机器程序

100001	10
00000111	
100010	10
00001010	
100101	11
00000110	
111101	00

所有程序都需转换成机器程序，计算机才能执行

计算机语言的发展

8

怎样解决机器语言编写程序所存在的困难----符号化语言/汇编语言

◆用符号编写程序 ==> 翻译 ==> 机器语言程序

◆人们提供了用助记符编写程序的规范/标准。同时开发了一个翻译程序（被称为汇编程序），实现了将符号程序自动转换成机器语言程序的功能

操作码	地址码
100001	1000000111



MOV A, 7



用助记符编写程序的语言，可与机器语言一一对应

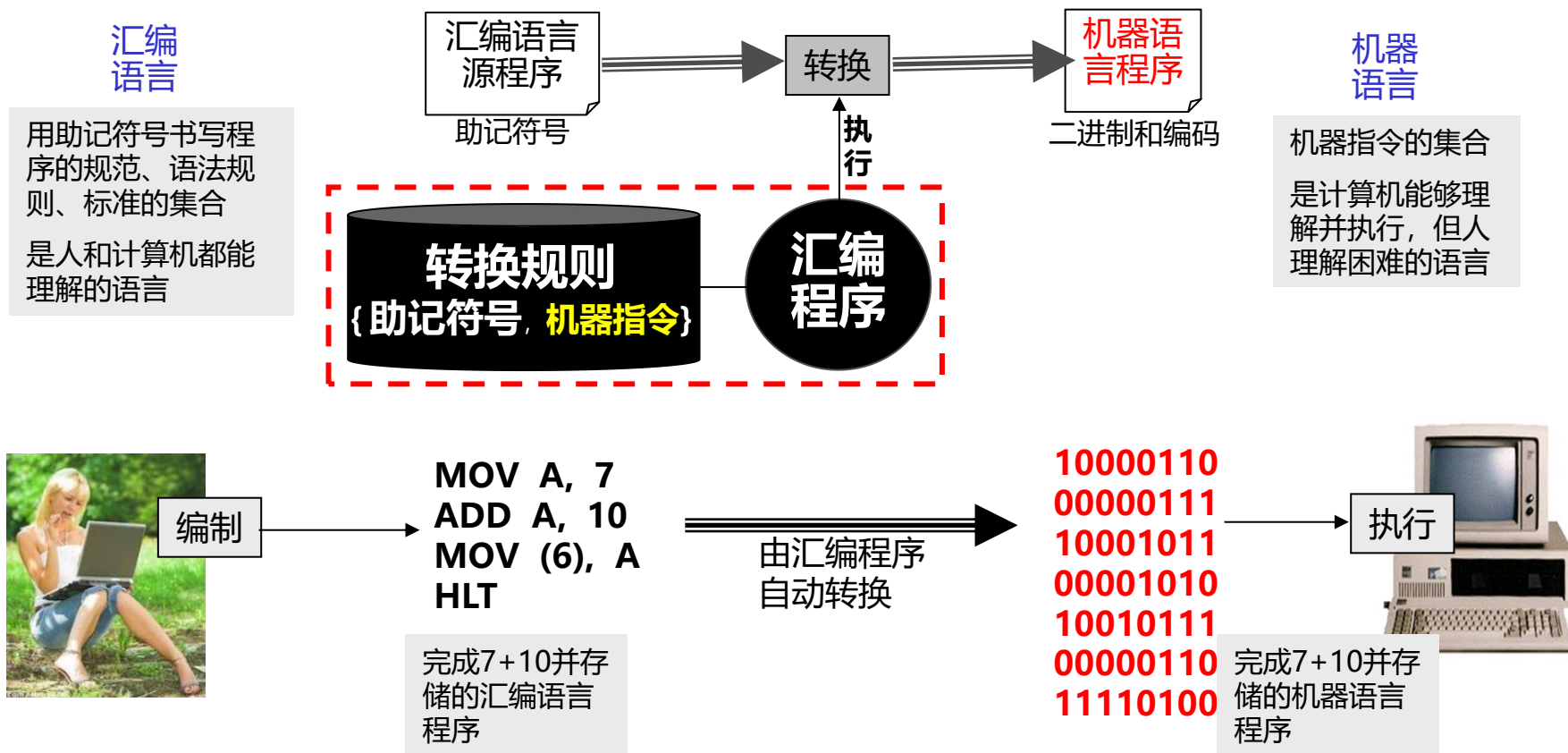
计算7+10并存储的汇编语言源程序

```
MOV A, 7
ADD A, 10
MOV (6), A
HLT
```


计算机语言的发展

9

符号化程序机器不能直接执行怎么办----汇编/翻译



测练题：关于汇编语言程序，下列说法**正确**的是_____。

- ☐ A 汇编程序，是用汇编语言编写的程序
- ☒ B 汇编程序，是将汇编语言源程序转换成机器程序的程序
- ☐ C 汇编程序，是能够被机器直接执行的程序
- ☐ D 汇编程序，是用助记符等编写的程序

提交

计算机语言的发展

11

高级语言

◆人们提供了类似于自然语言方式、以语句为单位书写程序的规范/标准。并开发了一个翻译程序，实现了将语句程序自动翻译成机器语言程序的功能。



用类似自然语言的语句编写程序的语言。

计算7+10并存储的高级语言(源)程序

高级语言源程序：是用高级语言编出的程序。

编译程序：是将高级语言源程序翻译成机器语言程序的程序。

```
Result = 7+10;  
Return
```

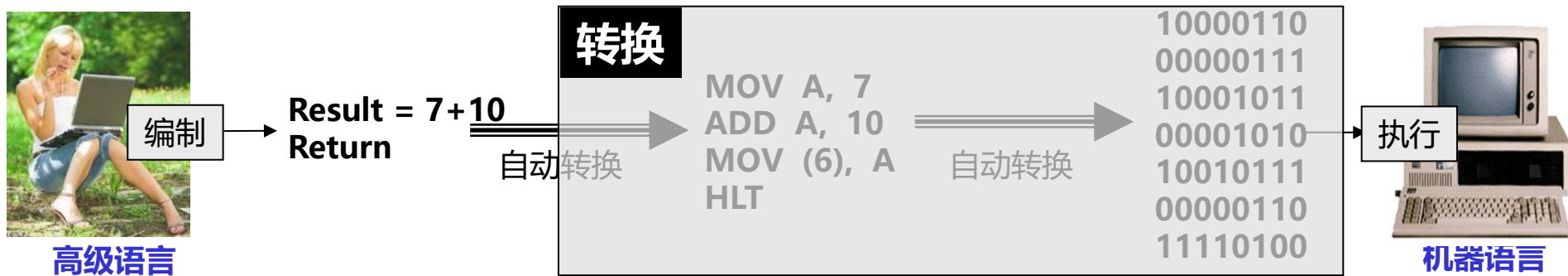
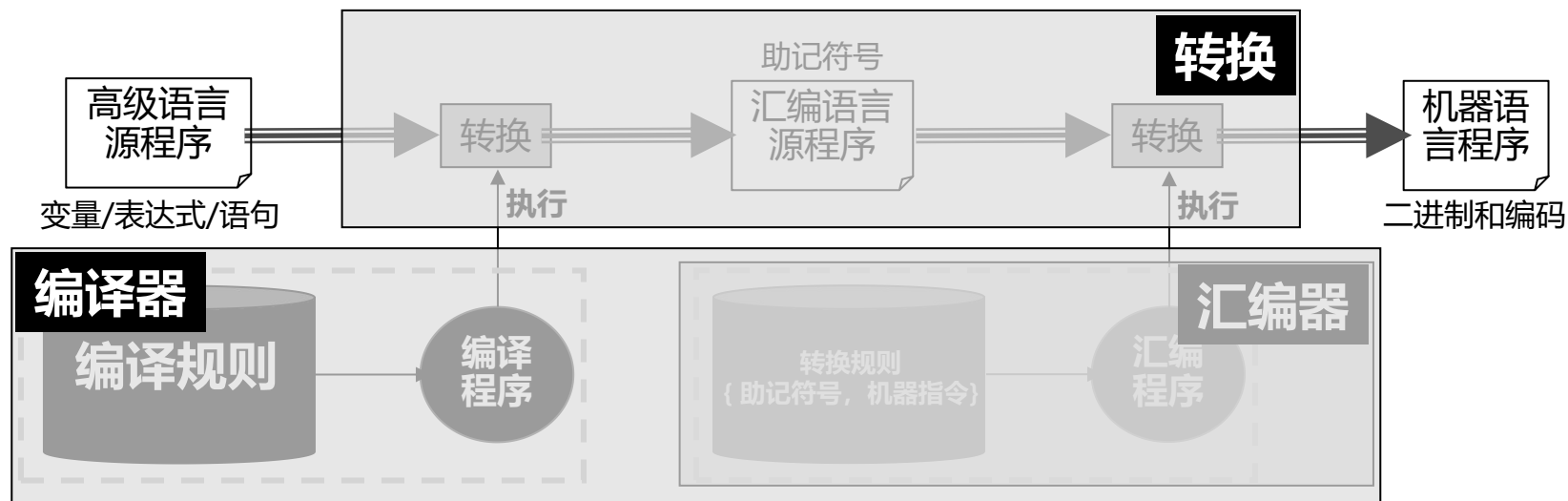
计算机语言的发展

12

高级语言 vs. 汇编语言

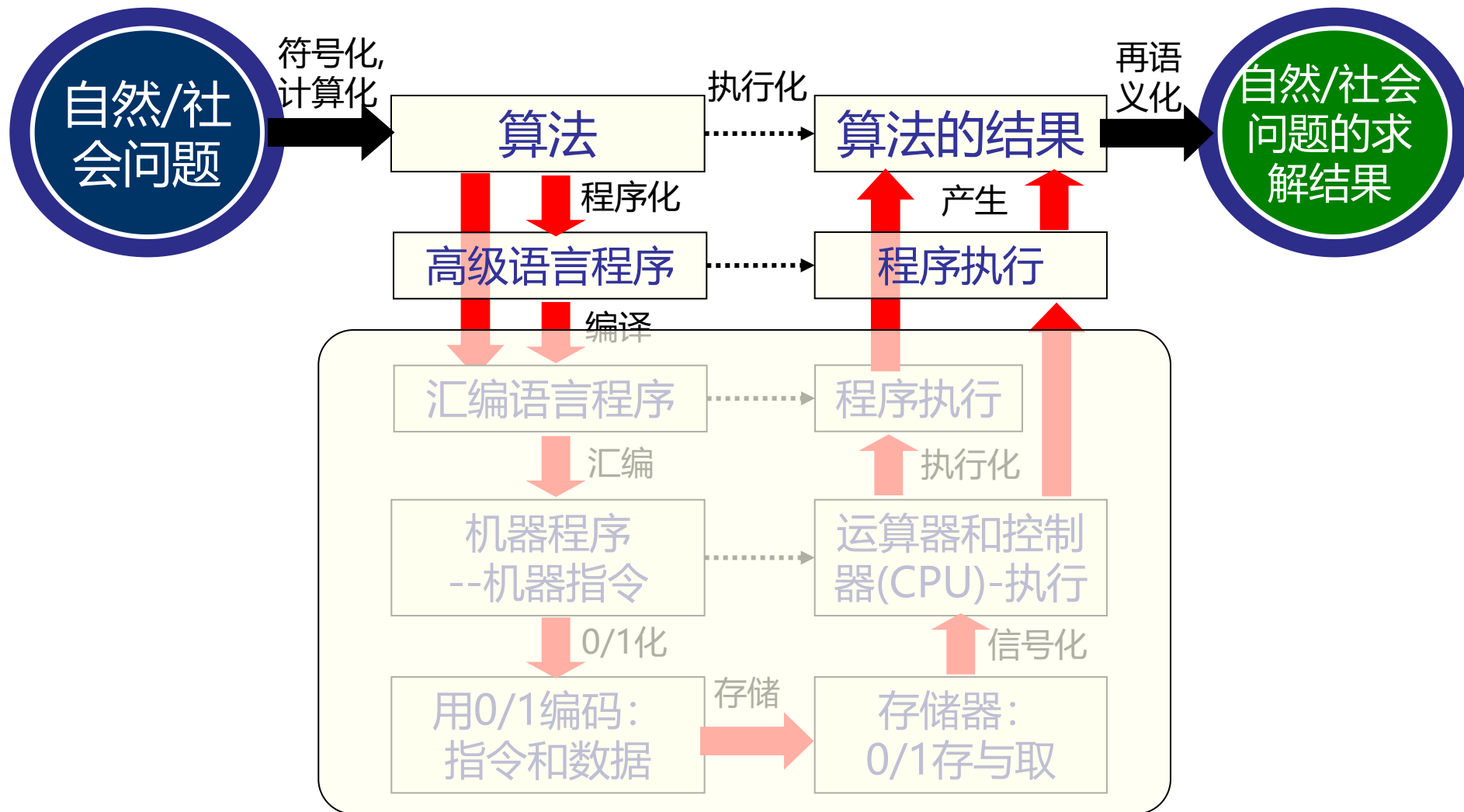
高级语言：机器无关性；一条高级语言语句往往可由若干条机器语言语句实现且不具有对应性

汇编语言：机器相关性；汇编语言语句和机器语言语句有对应性



计算机语言的发展

用高级语言进行问题求解的实现过程



计算机语言的发展

14

高级语言（源）程序

- 【符号化】 常量、变量—标识符
- 【语句化】 赋值、分支与循环
- 【结构化】 函数及其调用

【保留字】为编译器识别和处理语句而确定的一些特定符号；
【标识符】为常量或变量的名字

Python为例

变量

常量

保留字

k = **0**

for **k** **in** **range**(10):

print(**k**)

语句

表达式

怎样阅读高级语言程序：（1）识别标识符【常量、变量】；（2）识别表达式【算术表达式、比较表达式和逻辑表达式】，不管多么复杂，其结果就是一个值；（3）识别语句【赋值、分支和循环】；（4）识别函数及其调用。

上述程序，你
识别清楚了吗？

计算机语言的发展

15

高级语言程序的运行方式

机器语言编写的程序可以在计算机中直接运行，而汇编语言和高级语言编写的程序（通常称为源程序）则需要“翻译”成机器语言才能运行。

- 指源程序输入计算机后，翻译程序首先将整个程序翻译成用机器语言表示的目标程序，然后计算机再执行该目标程序，获得计算结果。
- 特点：运行时，计算机可以直接执行。
- 优势：运行速度快。
- 缺点：不能跨平台，开发效率低。

编译方式

- 指源程序进入计算机时，翻译程序逐条翻译程序指令，每翻译一条指令便立即执行。
- 特点：运行时逐语句解释执行。
- 优点：可以跨平台，开发效率高。
- 缺点：运行效率低。

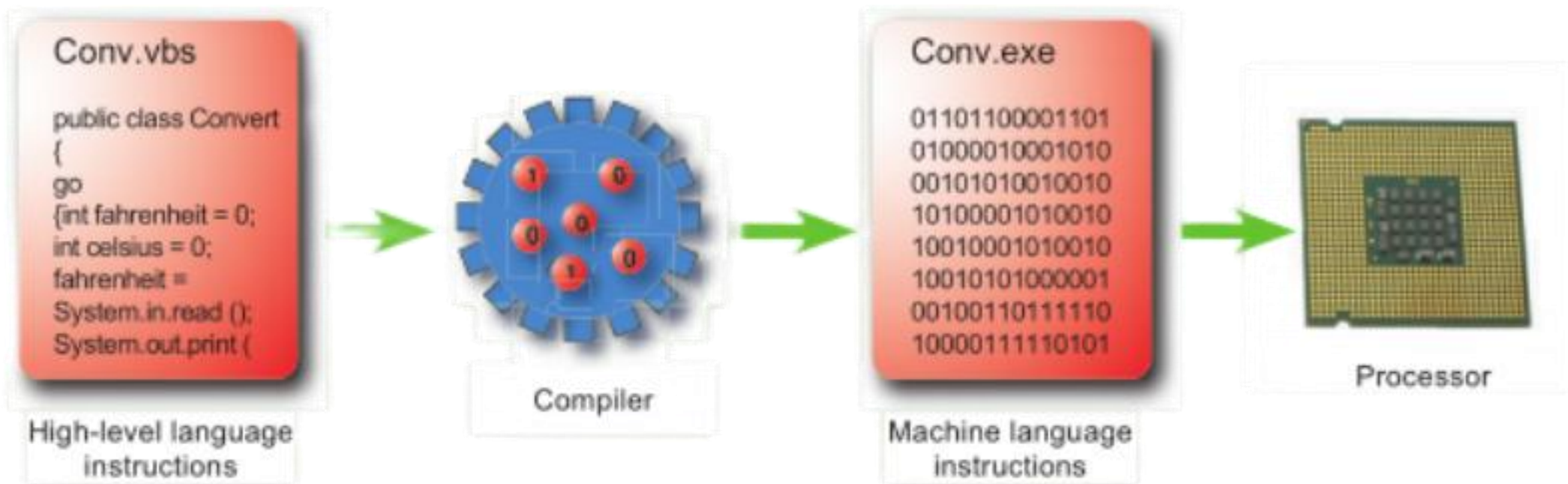
解释方式

计算机语言的发展

16

高级语言程序的运行方式

- **编译：** 编译器（**compiler**）在一个批处理中转换程序中所有语句，生成的指令集合称为目标代码（**object code**），被放置在一个新文件中

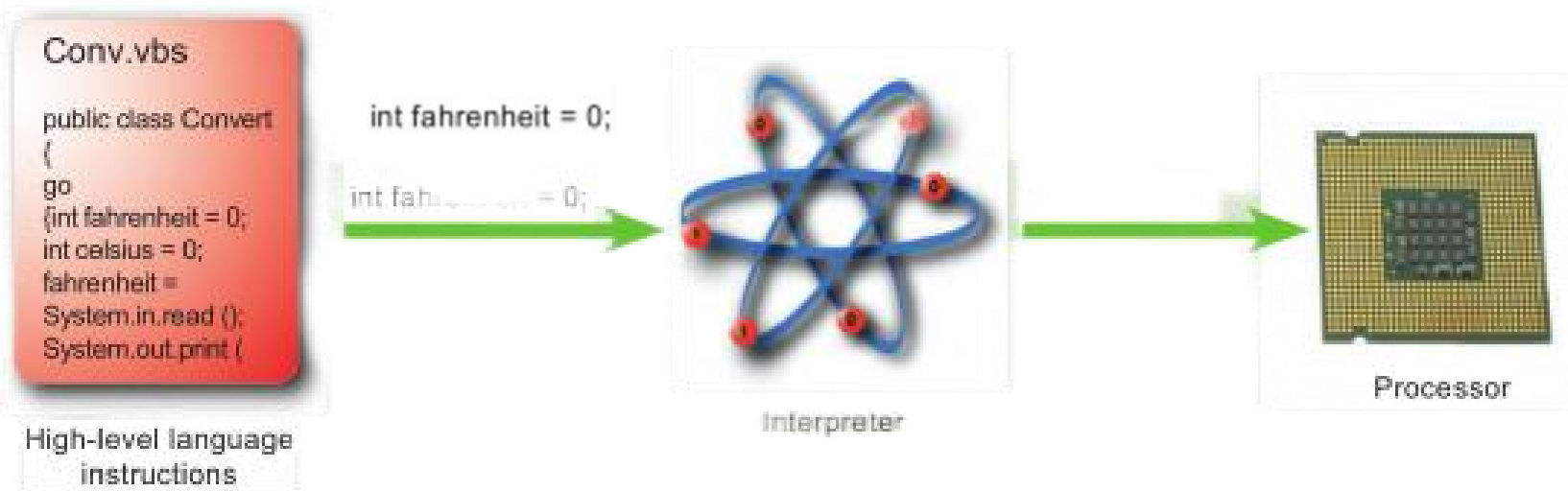


计算机语言的发展

17

高级语言程序的运行方式

- **解释：** 程序运行时，解释器（ **interpreter** ）一次转换并执行一条语句。执行完一条语句后，解释器转换并执行下一条语句，依此类推



计算机语言的发展

18

面向对象的程序设计语言与可视化构造语言

----像堆积木一样构造程序

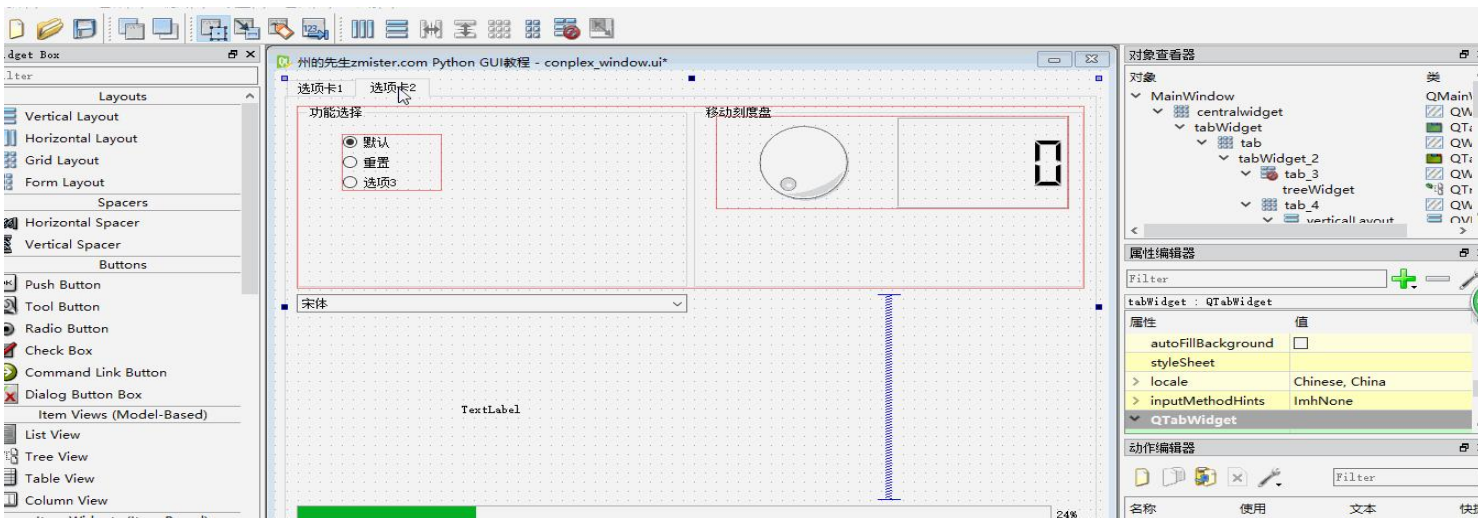
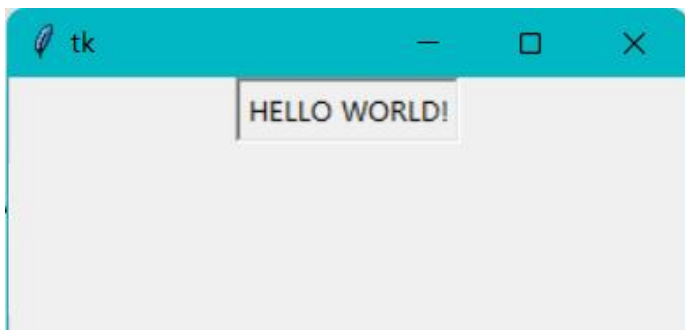
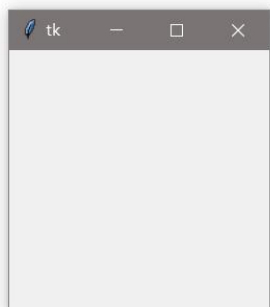
test_python.py - C:/Users/myj/Desktop/test_python.py (3.8.6)

File Edit Format Run Options Window Help

```
import tkinter
```

```
window=tkinter.Tk() #创建一个根窗口对象window
```

```
window.mainloop() #进入消息循环
```



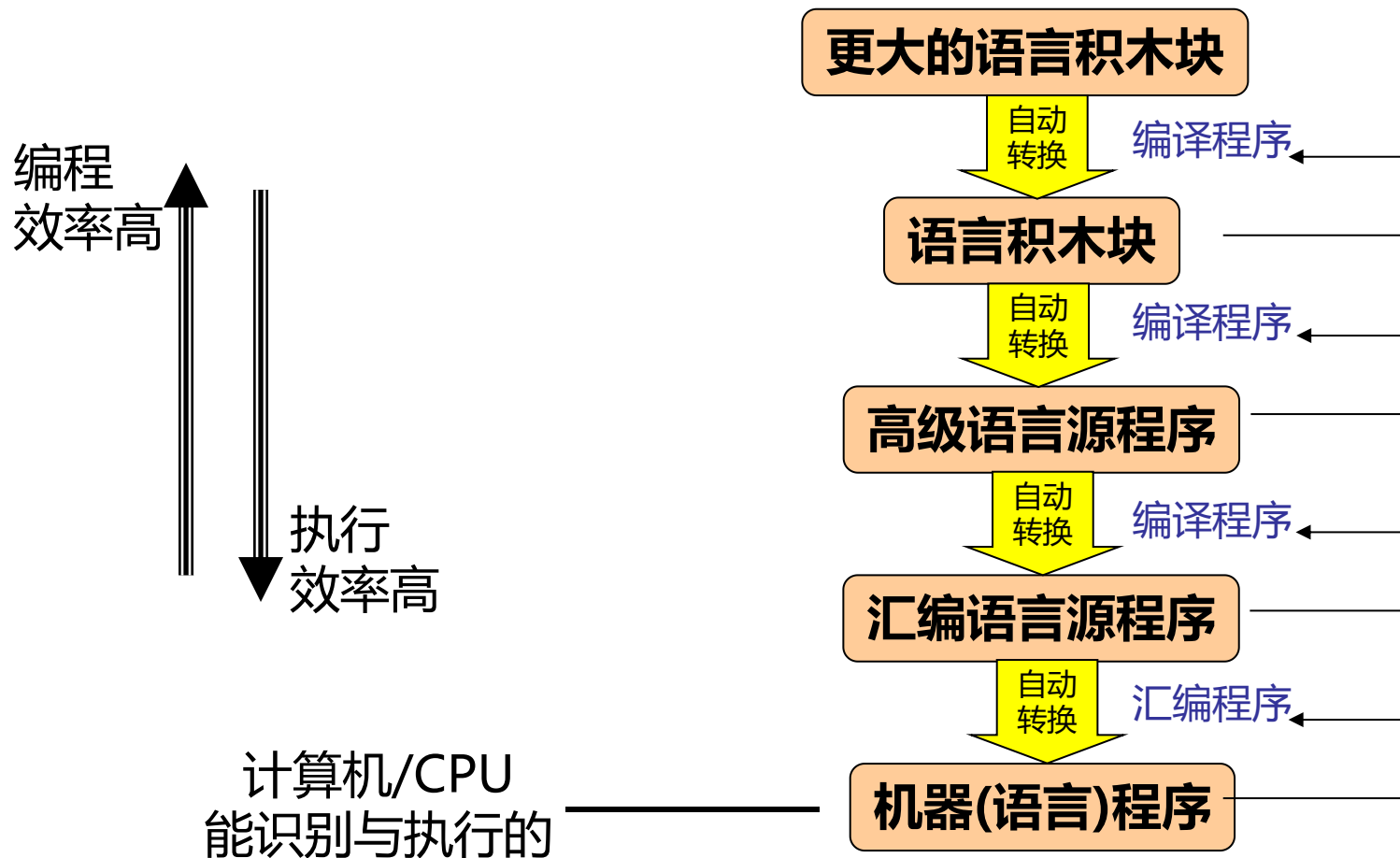
```
import tkinter

window = tkinter.Tk()
b = tkinter.Button(window, text="HELLO WORLD!", background="black")
b.pack()
window.mainloop()
```

计算机语言的发展

19

计算机语言发展的基本思维



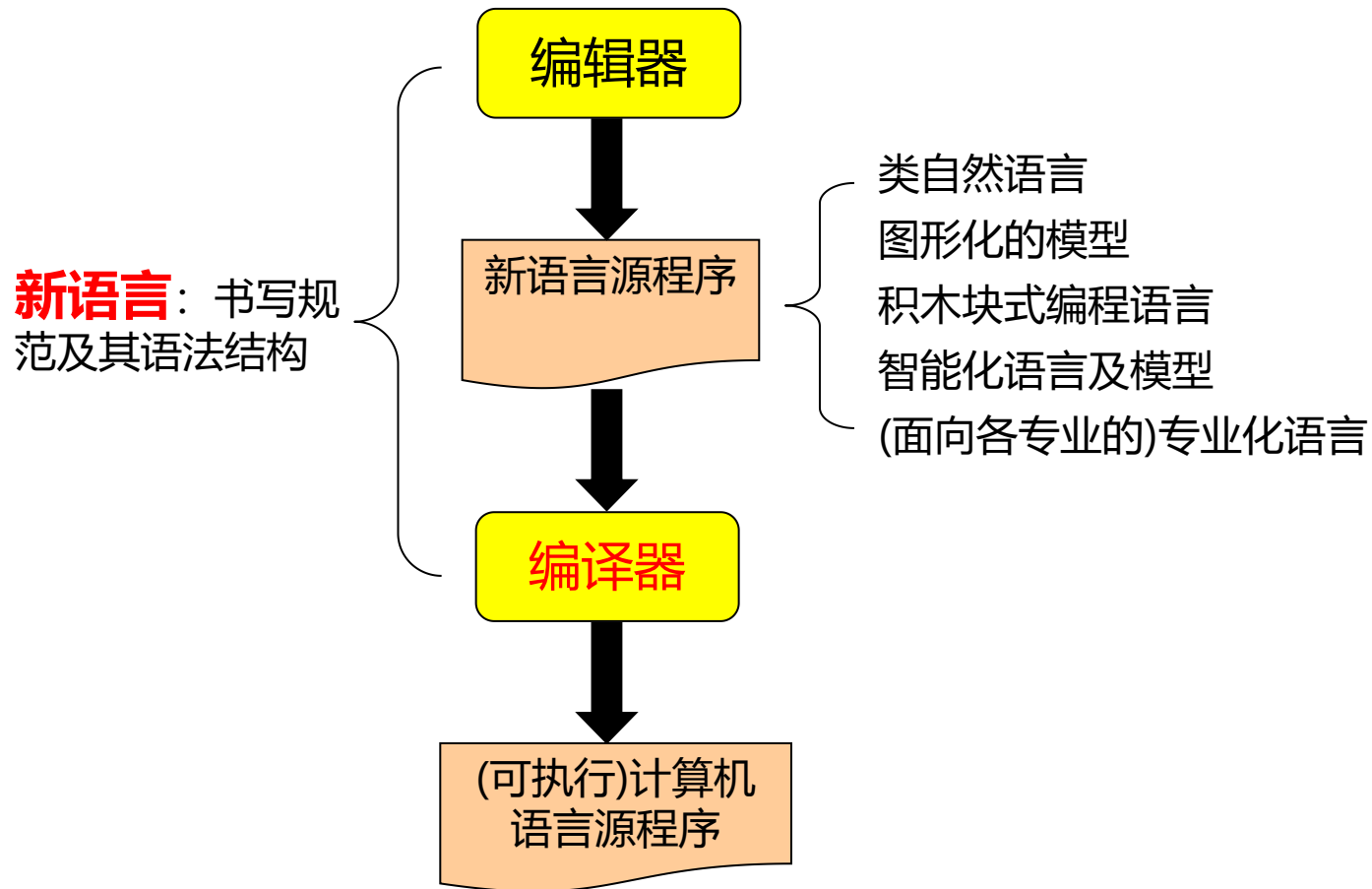
测练题：从机器语言，到高级语言，再到对象化编程语言，下列说法**正确**的是_____。

- ☐ A 编程效率越来越高，程序执行效率越来越高
- ☐ B 编程效率可能越来越低，程序执行效率越来越高
- ☒ C 编程效率越来越高，程序执行效率可能越来越低
- ☐ D 编程效率可能越来越低，程序执行效率可能越来越低

计算机语言的发展

21

不仅要用语言，还要发明新语言



计算机语言的发展

22

计算机技术是伴随着计算机语言的不断发展而发展起来的

因计算机语言
获得图灵奖的

- 1966 A.J. Perlis: 编程技术和编译架构
- 1972 E.W. Dijkstra: ALGOL语言
- 1974 Donald E. Knuth: 程序语言
- 1977 John Backus : 高级语言, Fortran
- 1979 Kenneth E. Iverson: 编程语言, APL
- 1980 C. Antony R. Hoare: 编程语言
- 1981 Edgar F. Codd: 关系数据库语言
- 1984 Niklaus Wirth: 开发了EULER、ALGOL-W、MODULA和PASCAL一系列崭新的计算语言。
- 1987 John Cocke: 编译器
- 2001 Ole-Johan Dahl、Kristen Nygaard: 面向对象编程, SIMULA I 和SIMULA 67中。
- 2003 Alan Kay :面向对象语言, Smalltalk
- 2005 Peter Naur: Algol60程序语言。
- 2006 Fran Allen: 编译器

计算机语言的发展

多种多样的高级语言

Mother Tongues

Tracing the roots of computer languages through the ages

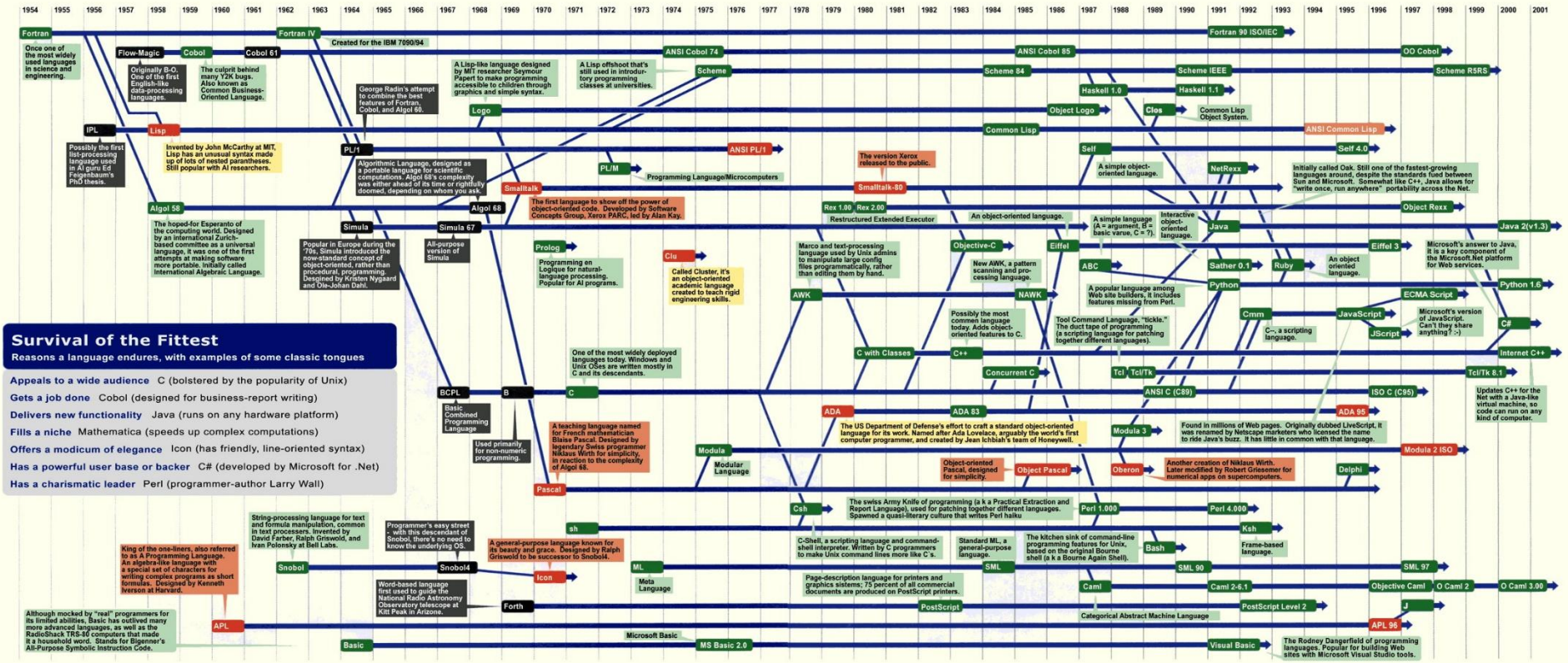
Just like half of the world's spoken tongues, most of the 2,300-plus computer programming languages are either endangered or extinct. As powerhouses C/C++, Visual Basic, Cobol, Java and other modern source codes dominate our systems, hundreds of older languages are running out of life.

An ad hoc collection of engineers-electronic lexicographers, if you will-aim to save, or at least document the lingo of classic software. They're combing the globe's 9 million developers in search of coders still fluent in these nearly forgotten lingua frangas. Among the most endangered are Ada, APL, B (the predecessor of C), Lsp, Oberon, Smalltalk, and Simula.

Code-raker Grady Booch, Rational Software's chief scientist, is working with the Computer History Museum in Silicon Valley to record and, in some cases, maintain languages by writing new compilers so our ever-changing hardware can grok the code. Why bother? "They tell us about the state of software practice, the minds of their inventors, and the technical, social, and economic forces that shaped history at the time," Booch explains. "They'll provide the raw material for software archaeologists, historians, and developers to learn what worked, what was brilliant, and what was an utter failure." Here's a peek at the strongest branches of programming's family tree. For a nearly exhaustive rundown, check out the Language List at [HTTP://www.informatik.uni-freiburg.de/Java/misc/lang_list.html](http://www.informatik.uni-freiburg.de/Java/misc/lang_list.html). - Michael Mendeno

Key

- 1954 Year introduced
- Active: thousands of users
- Protected: taught at universities; compilers available
- Endangered: usage dropping off
- Extinct: no known active users or up-to-date compilers
- Lineage continues

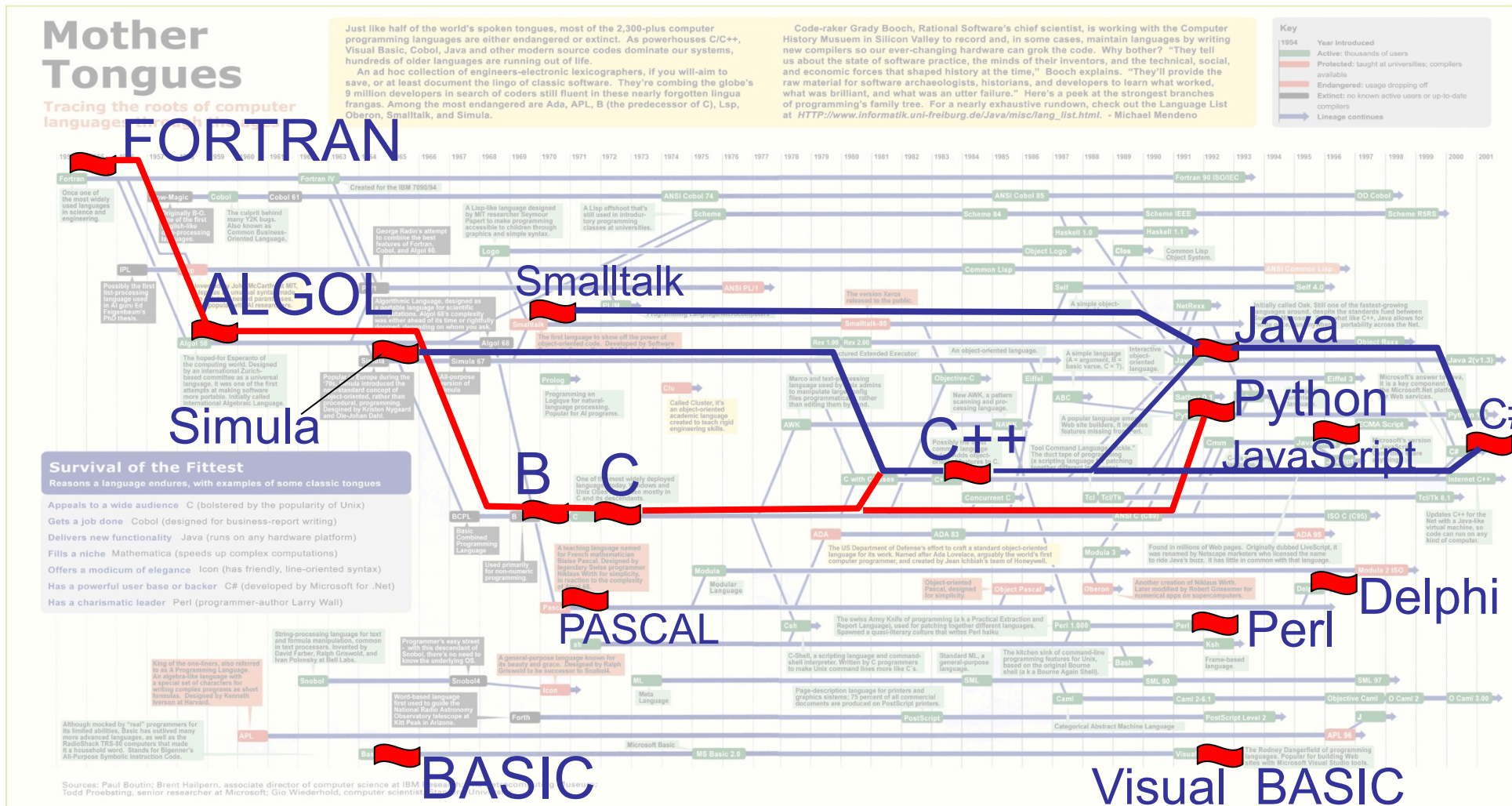


Sources: Paul Boutin; Brent Halpern, associate director of computer science at IBM Research; The Retrocomputing Museum; Todd Proebsting, senior researcher at Microsoft; Gio Wiederhold, computer scientist, Stanford University

计算机语言的发展

24

多种多样的高级语言



测练题：关于计算机语言，下列说法**不正确**的是_____。

- A 汇编语言和机器语言是以指令为单位来编写程序；
- B 高级语言是以语句为单位来编写程序，一条语句相当于若干条指令(或者说一条语句可用若干条指令来实现)；
- C 面向对象语言或可视化构造语言是以对象(类)为单位来编写程序，一个对象相当于若干条语句((或者说一个对象可用若干条语句来实现)；
- D 我们可以设计一种新语言，让用户以其更熟悉的对象(类)来编写源程序，然后提供一个编译器将该源程序转换成某种已广泛使用的高级语言源程序，就可以让机器执行该程序。
- E 上述说法有不正确的。

提交

第3章 程序设计与算法思维

26

- 一、计算机语言的发展**
- 二、不同的计算机语言之异同点**
- 三、程序构造：控制结构**
- 四、程序构造：函数、递归与迭代**
- 五、算法表达与思维**

不同的计算机语言之异同点

27

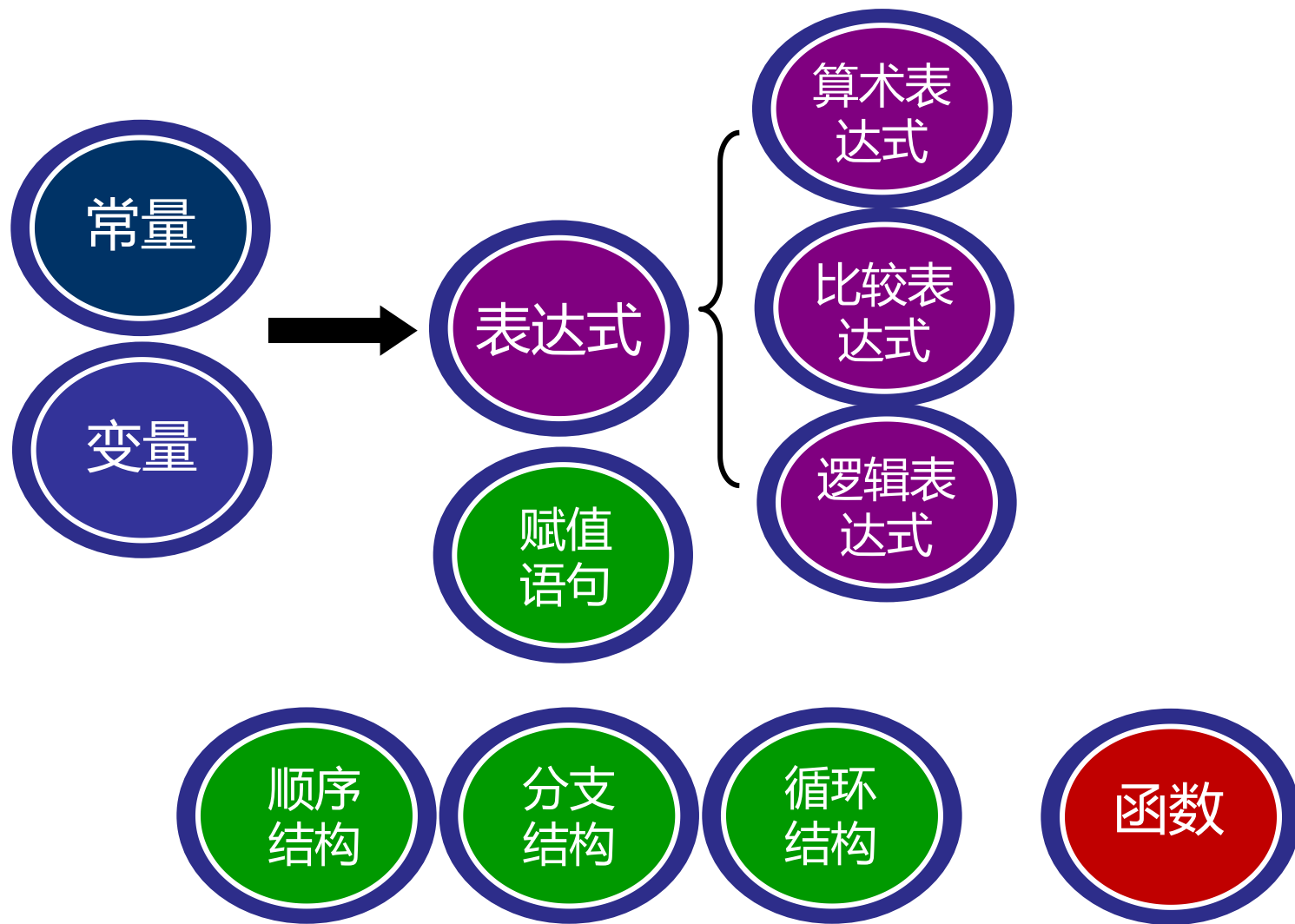
典型语言比较【C语言与Python语言】

//C 语言的冒泡排序程序

```
void bubble_sort(int *lists, int count)
{
    int i, j;
    for(i=0; i<count-1; i++)
    {
        for(j=0; j<count-i; j++)
        {
            if (lists[j] < lists[j+1])
            {
                int k = lists[j];
                lists[j] = lists[j+1];
                lists[j+1] = k;
            }
        }
    }
}
```

Python 语言的冒泡排序程序

```
def bubble_sort(lists, count):
    for i in range(0, count):
        for j in range(0, count-i):
            if lists[j] < lists[j+1]:
                k=lists[j];
                lists[j]=lists[j+1];
                lists[j+1]=k;
    return lists
```



不同的计算机语言之异同点

28

几种语言比较

//C 语言的冒泡排序程序

```
void bubble_sort(int *lists, int count)
{
    int i, j;
    for(i=0; i<count-1; i++)
    {
        for(j=0; j<count-i; j++)
        {
            if (lists[j] < lists[j+1])
            {
                int k = lists[j];
                lists[j] = lists[j+1];
                lists[j+1] = k;
            }
        }
    }
}
```

Python 语言的冒泡排序程序

```
def bubble_sort(lists, count):
    for i in range(0, count):
        for j in range(0, count-i):
            if lists[j] < lists[j+1]:
                k=lists[j];
                lists[j]=lists[j+1];
                lists[j+1]=k;
    return lists
```

基本程序要素 (本书)	Python	C	Visual Basic
算术表达式 +、-、*、/、**(幂)、mod(取模)	基本部分: 相同 +、-、*、/、**、%(取模)、//(取整)	基本部分: 相同 +、-、*、/、%(取模)	基本部分: 相同 +、-、*、/、^(幂)、mod(取模)
$a+b$ $a**3$ $9 \bmod 4$	$a+b$ $a**3$ $9 \% 4$	$a+b$ $9 \% 4$	$a+b$ a^3 $9 \bmod 4$
比较表达式/关系运算符 ==、<、>、<=、>=、<=	相同 ==、!=(<>)、>、<、>=、<=	基本部分: 相同 ==、!=(>、<、>=、<=	相同 ==、<、>、<=、>=、<=
$a > b$ $a < b$	$a > b$ $a != b$	$a > b$ $a != b$	$a > b$ $a < b$
逻辑运算符 and、or、not	相同 and、or、not	不同 &&、 、!	相同 and、or、not
$(a > b)$ and $(c > d)$	$(a > b)$ and $(c > d)$	$(a > b)$ && $(c > d)$	$(a > b)$ and $(c > d)$
赋值语句 数据类型 变量名; 变量 = 表达式;	相同: 可直接赋值 变量 = 表达式	相同: 变量需事先声明 数据类型 变量名; 变量 = 表达式;	相同: 变量需事先声明 Dim 变量名 AS 类型 变量 = 表达式;
int i, j; i = a + b;	i = a + b	int i; i = a + b	Dim i as integer i = a + b
If 条件 Then { 执行语句 11; 执行语句 12; } Else { 执行语句 2; }	If 条件: 执行语句 11 执行语句 12 Else: 执行语句 2 (注: 用等长缩进区分语句块)	If 条件 Then { 执行语句 11; 执行语句 12; } Else { 执行语句 2; }	If 条件 Then 执行语句 11; 执行语句 12; Else 执行语句 2 EndIf
If $(a > b)$ Then { c=a; a=b; } Else { b=a; a=c; }	If $a > b$: c=a a=b Else b=a a=c	If $(a > b)$ Then { c=a; a=b; } Else { b=a; a=c; }	If $(a > b)$ Then c=a a=b Else b=a; a=c; EndIf
For 计数器=起始值 to 结束值 { 循环语句 1; 循环语句 2; }	for 计数器 in range(下界值, 上界值): 循环语句 1 循环语句 2 (注: 用等长缩进区分语句块)	For (计数器=初值; 条件; 更新) { 循环语句 1; 循环语句 2; }	For 计数器 = 起始值 to 结束值 循环语句 1; 循环语句 2; Next 计数器

不同的计算机语言之异同点

29

几种语言比较

//C 语言的冒泡排序程序

```
void bubble_sort(int *lists, int count)
{
    int i, j;
    for(i=0; i<count-1; i++)
    {
        for(j=0; j<count-i; j++)
        {
            if (lists[j] < lists[j+1])
            {
                int k = lists[j];
                lists[j] = lists[j+1];
                lists[j+1] = k;
            }
        }
    }
}
```

Python 语言的冒泡排序程序

```
def bubble_sort(lists, count):
    for i in range(0, count):
        for j in range(0, count-i):
            if lists[j] < lists[j+1]:
                k=lists[j];
                lists[j]=lists[j+1];
                lists[j+1]=k;
    return lists
```

基本程序要素 (本书)	Python	C	Visual Basic
For a = 10 to 20 Step 1 { Sum = Sum + a; }	For a in range(10, 20): Sum = Sum + a	For(int a = 10; a < 20; a = a + 1) { Sum = Sum + a; }	For a = 10 to 20 Step 1 Sum = Sum + a; Next a
While (条件) { 循环语句 1; 循环语句 2; }	While 条件: 循环语句 1 循环语句 2 (注: 用等长缩进区分语句块)	While (条件) { 循环语句 1; 循环语句 2; }	Do While 条件 循环语句 1; 循环语句 2; Loop
While (a < 20) { Sum = Sum + a; a = a - 1; }	While a < 20: Sum = Sum + a a = a - 1	While (a < 20) { Sum = Sum + a; a = a - 1; }	Do While (a < 20) Sum = Sum + a a = a - 1 Loop
类型 函数名(Para) { 函数体; } 变量 = funcname(para)	def 函数名(para): 函数体 return [返回值] (注: 用等长缩进区分语句块) 变量 = funcname(para)	类型 函数名(Para) { 函数体; } 变量 = funcname(para)	Function 函数名(Para) As 类型 函数体 End Function 变量 = funcname(para)
Long Fib(int n) { Fib = 120; } ret = Fib(5);	def Fib (n): Fib = 120 ret = Fib(5)	Long Fib(int n) { Fib = 120; } ret = Fib(5);	Function Fib(n As Integer) As Long Fib = 120 End Function ret = Fib(5)

第3章 程序设计与算法思维

30

- 一、计算机语言的发展
- 二、不同的计算机语言之异同点
- 三、程序构造：控制结构
- 四、程序构造：函数、递归与迭代
- 五、算法表达与思维

程序构造：控制结构

31

程序结构：以Python为例

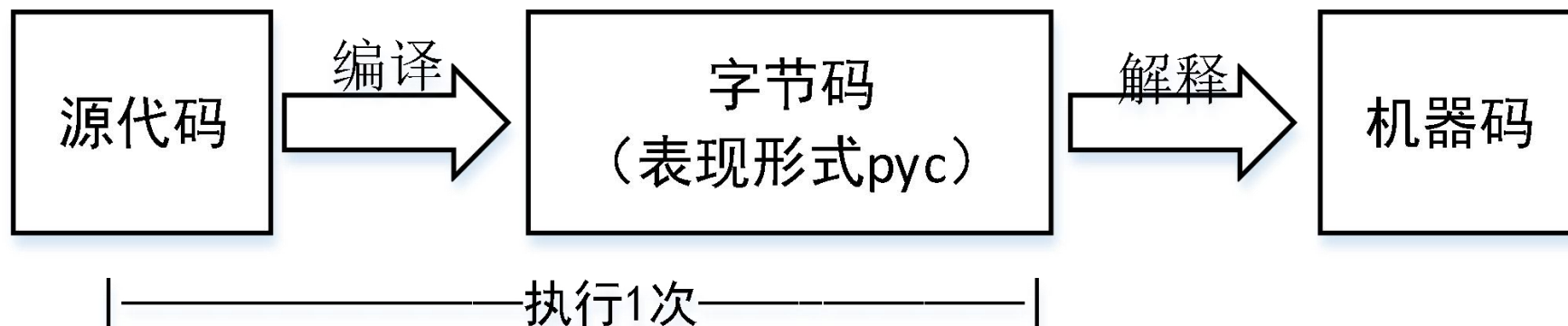
- ✓ Python的创始人是荷兰的吉多·范罗苏姆 (Guido van Rossum)
- ✓ Python语言诞生于1989年，第1个公开发行人版本发行于1991年
- ✓ Python语言有以下主要的优势：
 - (1) 语法简洁而清晰，代码的可读性高
 - (2) 开发效率高
 - (3) 跨平台特性。Python可以真正做到跨平台，开发的程序可以运行在Windows、Linux、MacOS系统下
 - (4) 大量丰富的库或扩展。
 - (5) 代码量少，一定程度上提高了软件质量

程序构造：控制结构

32

程序结构：以Python为例

从计算机的角度看，Python程序的运行过程包含两个步骤：解释器解释和虚拟机运行。



- Python还可以通过交互方式运行。例如，在UNIX/Linux、Mac、Window等系统的命令模式下运行Python交互环境，然后输入Python指令直接运行。
- 实际开发中，python常被昵称为胶水语言，能够很轻松的把用其他语言制作的各种模块（尤其是C/C++）轻松地联结在一起。

程序构造：控制结构

33

程序结构：以Python为例

Python语言程序设计的基本元素包括：常量、变量、关键字、运算符、表达式、函数、语句、类、包与模块等。

常量

初始化（第一次赋予值）后就保持固定不变的值

变量

在运行过程中值可以被修改的量

- 使用有意义的名字
- 多个单词之间为表示区隔，可以用下划线来连接,或者把每个单词的首字母大写
- 固定不变的常量名称一般用全大写英文字母
- 一般不用下划线作为开头字符
- 严格区分大小字母

程序构造：控制结构

34

程序结构：以Python为例

运算符

用来对数据进行
操作的符号

赋值、算术、比较、逻辑

表达式由常量、变量加运算符构成，如 $1+2$

函数

相对独立的功能
单位，可以执行
一定的任务

语句

语句是由表达式、
函数调用组成的

如 $x=1+2$

类

同一类事物的抽
象

模块

把一组相关的名
称、函数、类
(或者是) 它们
的组合组织到一个文件中。

包

由一系列模块组
成的集合，包是
一个有层次的文
件目录结构

程序构造：控制结构

35

基本要素：（1）常量、变量与（2）表达式

常量、变量 与表达式

◆算术表达式示例。算术表达式的结果是一数值；

$$A1 + (B2 - x1 + 76) * 3$$
$$(B2 + yy4) / L3 - xx3$$

◆比较表达式示例。比较表达式的计算结果是逻辑“真”或“假”；

$$\text{Grade} < 90$$
$$\text{Grade} \geq 70$$
$$N4 < A1 + B2 + 20 \quad // \text{注：} A1+B2+20 \text{为算术表达式，计算完后再与} N4 \text{的值进行比较}$$

◆逻辑表达式示例。逻辑表达式的计算结果是逻辑“真”或“假”；

$$(x1 \geq A1) \&\& (B2 \neq y2)$$

◆将表达式的计算结果赋值给一变量：赋值语句

$$M = X > Y + 50;$$
$$M = (X > Y) \text{ AND } (X < Y);$$
$$K = K + (5 * K);$$

程序构造：控制结构

36

程序结构：以Python为例

赋值语句

Python中赋值语句的本质是创建一个对象的引用

变量**赋值**可以通过等号(=)来实现，其语法格式：

变量名 = 变量值

例如，`myvalue = 123`

例如，`myvalue = "学习强国"`

例如，`a=b=c=1`

例如，`a, b, c = 11, 23, 'python'`

数值型的变量

字符串类型的变量

同时为多个变量赋值

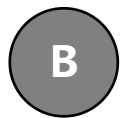
为多个对象指定多个变量

测练题：已知 $X=21$ ， $Y=15$ ， $Z=22$ ，计算表达式 $((X>Y) \text{ or } (Y>Z)) \text{ and } ((X<Y) \text{ or } (Y<Z))$ 的值，结果为_____。



A

真



B

假

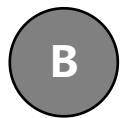
提交

测练题：已知 $X=21$ ， $Y=15$ ， $Z=22$ ，计算表达式 $((X>Y) \text{ AND } (Y>Z))$ OR $((X<Z) \text{ AND } (Y<Z))$ 的值，结果为_____。



A

真



B

假

提交

程序构造：控制结构

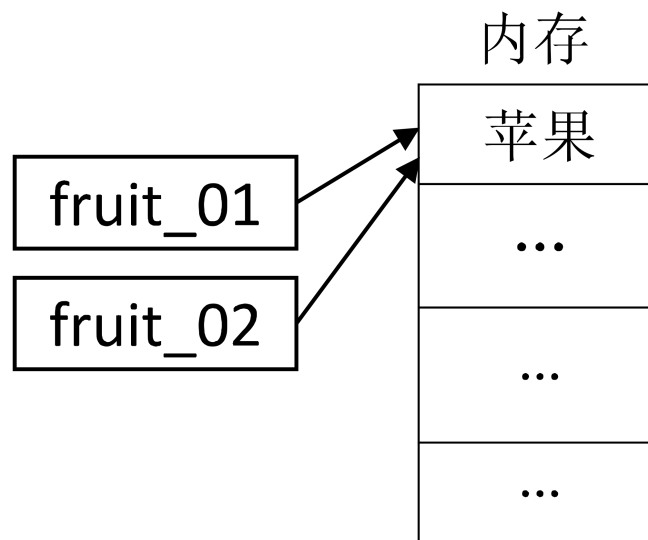
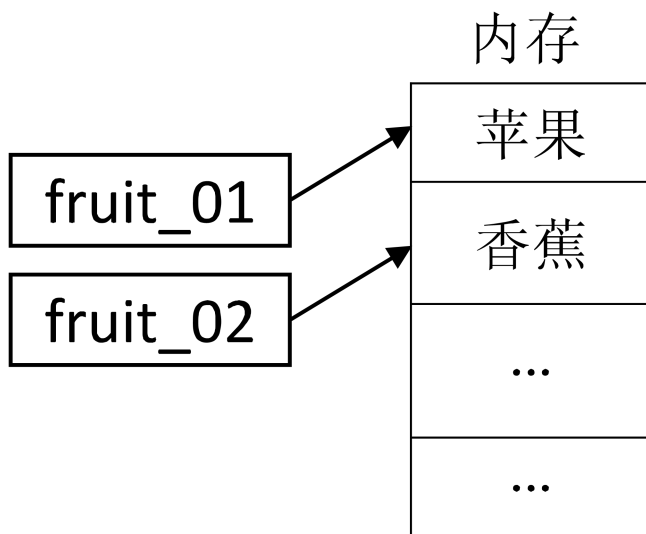
39

程序结构：以Python为例

变量fruit_01赋值“苹果”，变量fruit_02赋值“香蕉”，代码如下：

```
fruit_01 = '苹果'  
fruit_02 = '香蕉'
```

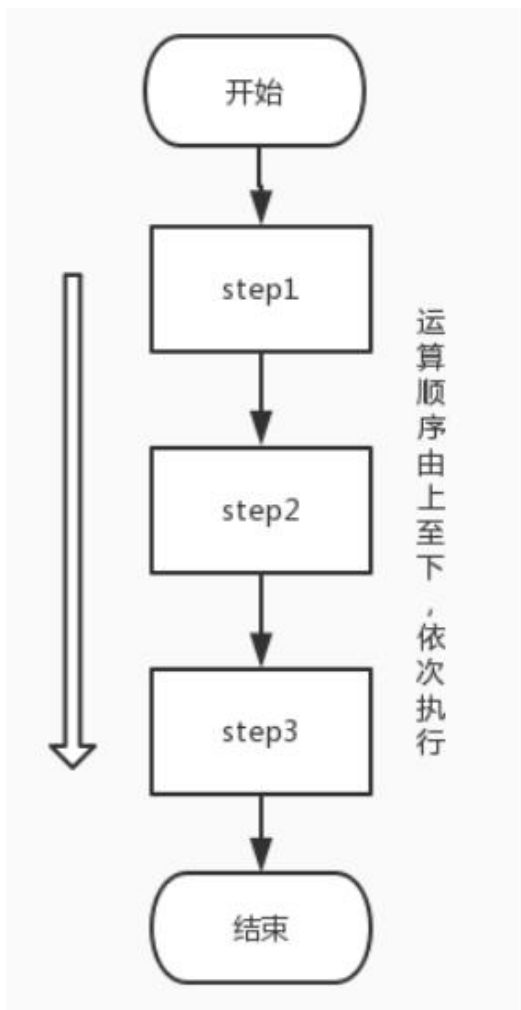
```
fruit_01 = '苹果'  
fruit_02 = fruit_01
```



程序构造：控制结构

40

顺序结构



```
a = 10
b = 20
c = 30
c = a + b
b = c - a
a = b + c
```

```
a = 10
b = 20
c = 30
b = c - a
a = b + c
c = a + b
```

#以Python语言为例

两组表达式的结果一样么？

语句从上到下，从左到右的顺序执行

程序构造：控制结构

41

顺序结构

交换a和b

```
a = 10
b = 20
tmp = b
b = a
a = tmp
```

最适宜运动心率 = $(220 - \text{年龄} - \text{安静心率}) \times (60\% \sim 80\%) + \text{安静心率}$

输入：第一行输入年龄，第二行输入安静心率

输出：输出最适宜的运动心率的范围

```
a=float(input())
b=float(input())
l=(220-a-b)*0.6+b
h = (220-a-b)*0.8+b
print(l, h, sep='~')
```

#以Python语言为例

程序构造：控制结构

42

分支结构

Python 支持来自数学的常用逻辑条件：

- 等于： **`a == b`**
- 不等于： **`a != b`**
- 小于： **`a < b`**
- 小于等于： **`a <= b`**
- 大于： **`a > b`**
- 大于等于： **`a >= b`**

```
success = input("请输入您的成绩： ")
success = int(success)
if success >= 60:
    print("及格")
else:
    print("不及格")
```

```
x = 52
if x > 10:
    print("Above ten,")
    if x > 20:
        print("and also above 20!")
    else:
        print("but not above 20.")
```

#以Python语言为例

高级语言程序的基本要素

43

分支结构

分支语句

If (条件表达式)

{ 条件表达式为真时执行的语句序列 }

Else

{ 条件表达式为假时执行的语句序列 }

X	500
Y	50
Z	80

```
Y = 50;  
Z = 80;  
X = 30;  
X = Z + Y;  
If Y > Z {  
    X = X - Y; }  
Else {  
    X = X - Z; }  
X = X + Y;  
If X > Z { X = Y; }  
X = X - Z;  
If X > Y  
{ X = X - Y; }
```

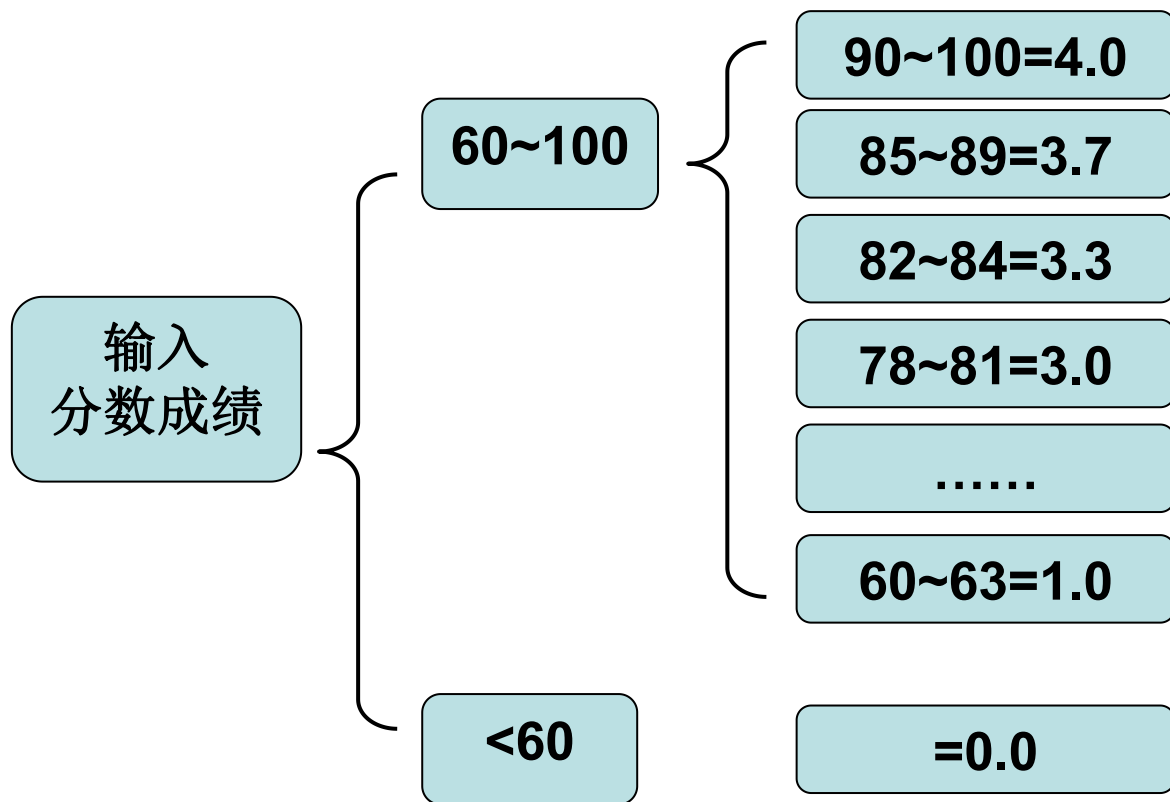
程序构造：控制结构

44

分支结构

对于多层级的分支结构，如何实现？

如：设计一段程序实现分数成绩和绩点数的换算



```
success = int(input("请输入您的成绩: "))
if success < 60:
    print("GPA=0.0")
elif success >= 60 and success <=100:
    if success >= 60 and success <= 63:
        print("GPA=1.0")
    .....
    elif success >= 85 and success < 90:
        print("GPA=3.7 ")
    elif success >= 90 and success <= 100 :
        print(" GPA=4.0 ")
else:
    print("您是火星来的吗? ")
```

#以Python语言为例

程序构造：控制结构

45

循环结构

循环语句(条件循环语句)

Do
 { 循环体的程序语句序列 }
While (条件表达式);

X	1
Y	1
Sum	01

```
X=1;  
Y=2;  
Sum=0;  
Do {  
    Sum = X+Y;  
    X=X+1;  
    Y=Y+1;  
} While (Sum<=10)  
//其他语句
```

程序构造：控制结构

46

循环结构

循环语句(条件循环语句)

While (条件表达式)

Do { 循环体的程序语句序列 }

X	1
Y	2
Sum	0

X=1;

Y=2;

Sum=0;

While (Sum<0)

Do {

Sum = X+Y;

X=X+1;

Y=Y+1;

}

<其他语句>

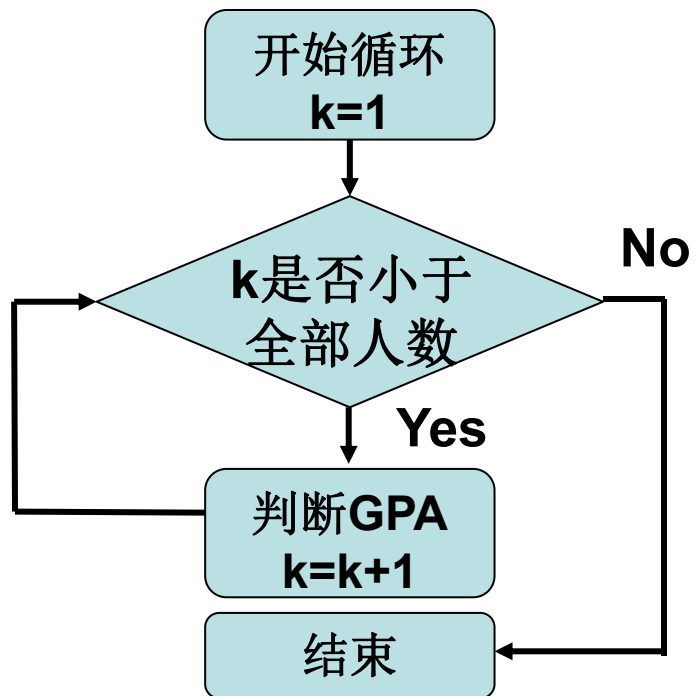
程序构造：控制结构

47

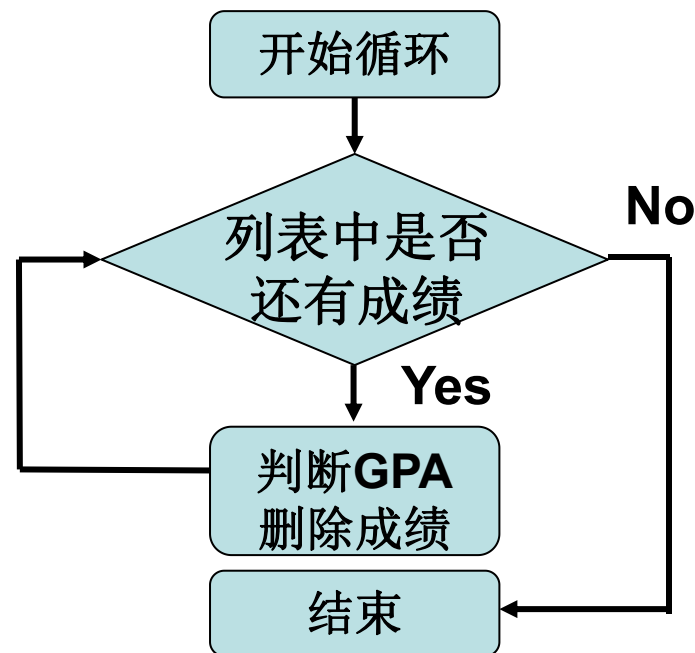
循环结构

对于实现分数成绩和绩点数换算的程序，如何把全班的成绩都完成绩点换算呢？

分支结构只可以做一件事，而循环结构加上分支结构则可以把这件事在短时间内做千千万万遍。只有掌握循环结构，才具备批量处理数据的能力。



一个一个依次处理循环体中所有元素



只有满足终止循环的条件才会退出循环

程序构造：控制结构

48

循环结构

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    if x == "banana":
        continue
    print(x)
```

```
C:\Users\My Name>python python_for_continue.py
apple
cherry
```

```
i = 1
while i < 6:
    print(i) i += 1
else:
    print("i is no longer less than 6")
```

```
C:\Users\My Name>python python_while_else.py
1
2
3
4
5
i is no longer less than 6
```

#以Python语言为例

测练题：关于普通计算机语言(或者说程序)的基本构成要素，下列说法最完整的是_____。

- ☐ A 常量与变量和表达式；
- ☐ B 常量与变量、表达式和语句；
- ☒ C 常量与变量、表达式、语句和函数；
- ☐ D 都不完整。

测练题：读右面的程序，并回答问题：程序行(60)执行了多少次？次数为_____。

A 4

B 5

C 6

D 7

```
(10)      N = 6;  
(20)      X = 0;  
(30)      Y = 1;  
(40)      For I = 1 To N-1 Step 1  
(50)          Z = X + Y;  
(60)          X = Y;  
(70)          Y = Z;  
(80)      Next I
```

提交

程序构造：函数

51

函数定义与调用

函数是一种抽象，用一个名字代表一个程序段落

